

Leveraging Quantum Annealing for Materialized View Selection

Immanuel Trummer

Cornell University
Ithaca, New York, USA
it224@cornell.edu

Abstract

The goal of materialized view selection is to select views that optimally support a given workload, while complying with storage constraints. This paper presents the first approach for solving materialized view selection on a quantum annealer. It shows how instances of materialized view selection can be transformed into semantically equivalent QUBO problems that can be solved on quantum annealers. The solution translates into an optimal choice of candidate views. The paper concludes with an analysis of the asymptotic growth of the required number of qubits as a function of the problem dimensions, and points out future work directions.

CCS Concepts

• **Information systems** → **Autonomous database administration**; *Query optimization*; • **Hardware** → **Quantum computation**.

Keywords

Quantum annealing, view selection, physical design

ACM Reference Format:

Immanuel Trummer. 2026. Leveraging Quantum Annealing for Materialized View Selection. In *The third workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '26)*, May 31–June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3811628.3811830>

1 Introduction

Quantum computing started out as a purely theoretical concept. Over the past years, it has transformed into a technology that is useful in practice (e.g., for drug discovery [34], transport logistics [28], and satellite positioning [33]), and more and more available to the general population (e.g., via cloud offerings such as D-Wave Leap¹ and Amazon Braket²). This makes it interesting to explore how this emerging class of hardware devices can be applied to solve hard problems that arise in the domain of data management. This paper focuses on a classical database optimization problem, materialized view selection [7, 8, 32], and describes how to leverage quantum accelerators to solve it.

The paper focuses on exploiting quantum annealers [3, 11, 20], a specific class of quantum computers. The number of qubits, often the limiting factor when solving problems of practical relevance,

tend to be higher for current annealer models (reaching up to 5,000 qubits for the latest D-Wave annealer models), compared to gate-based hardware [4, 9]. This motivates the focus on quantum annealers. In addition to quantum hardware, several classical accelerators such as Fujitsu’s digital annealer [16], NEC’s vector annealing machine [23], and Toshiba’s Simulated Bifurcation Machine [13] solve the same optimization problems addressed by quantum annealers, using classical hardware. The approach presented in this paper can be used equally for quantum as well as digital annealers.

Quantum annealers (and digital annealers) solve optimization problems that comply with a restrictive format: Quadratic Unconstrained Binary Optimization (QUBO).

Definition 1.1 (QUBO). An instance of quadratic unconstrained binary optimization is defined by a set $V = \{v_i\}$ of binary variables, and weights e_{ij} for $i \leq j$. An optimal solution assigns each variable v_i to a value a_i such that $\sum_{i \leq j} v_i \cdot v_j \cdot e_{ij}$ becomes minimal.

Note that this problem model supports formulas that contain quadratic terms (weighted by factors e_{ij} with $i \neq j$) as well as linear terms (weighted by factors e_{ij} with $i = j$).

To exploit quantum annealers for solving various optimization problems, we must first transform those problems into the QUBO format. The goal is to obtain a QUBO problem for which the optimal solution can be mapped back to an optimal solution to the original problem. In this paper, we will see an approach that maps instances of materialized view selection to an instance of QUBO. After transformation, an optimal QUBO solution can be transformed back into an optimal choice of materialized views.

The QUBO representation introduces binary variables that represent, for instance, the generation of one specific view (out of a set of candidates). Problem constraints must be represented by terms in the QUBO formula. If problem constraints are violated by a given value assignment to variables, those terms must take high values to discourage associated QUBO solutions. For materialized view selection, such QUBO terms represent, for instance, dependencies between views used and generated (discouraging solutions that use views that are not materialized to speed up queries), as well as storage constraints (penalizing solutions selecting views for materialization with an accumulated storage consumption above the user-defined storage threshold). Additional QUBO terms represent processing costs when using specific views for specific queries. As the goal of QUBO is to minimize the value of the QUBO formula, such terms motivate solutions that minimize processing costs.

Typically, the primary bottleneck preventing users from mapping problems of practically relevant sizes to quantum annealers is the limited number of qubits. Hence, it is crucial to find QUBO formulas that minimize the number of qubits required for representing problem instances. This paper analyzes how the asymptotic number of qubits required to represent the QUBO formula grows as a function of the input problem dimensions.

¹<https://www.dwavequantum.com/solutions-and-products/cloud-platform/>

²<https://aws.amazon.com/braket/>



This work is licensed under a Creative Commons Attribution 4.0 International License. *Q-Data '26, Bengaluru, India*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2703-0/26/05

<https://doi.org/10.1145/3811628.3811830>

The analysis distinguishes quantum annealers from digital annealers for the following reason. For quantum annealers, the connectivity between qubits is typically sparse, meaning that each qubit is only connected to a constant number of neighbors. Quadratic terms in the QUBO formula must be mapped to connections between qubit pairs. As the number of connections per qubit is a constant, the number of quadratic terms implies a lower bound on the number of required qubits (as, for QUBO variables that appear in many quadratic terms, we must represent them by multiple connected qubits on the annealer hardware to represent all quadratic terms involving the variable). For digital annealers, all qubits are typically connected, meaning that a single qubit suffices to represent a QUBO variable. Here, the number of required qubits is given directly by the number of QUBO problem variables.

In summary, the original scientific contributions in this paper are the following:

- The paper introduces a QUBO representation for the materialized view selection problem.
- The paper analyzes how the asymptotic number of required qubits grows as a function of the problem dimensions, distinguishing digital from quantum annealers.
- The paper describes future work direction on exploiting quantum computers for materialized view selection.

The remainder of this paper is organized as follows. Section 2 introduces the materialized view selection problem and related terminology. Section 3 shows how to transform instances of materialized view selection into semantically equivalent QUBO problems. Section 4 proves that the problem variant studied in this paper is NP-hard and analyzes the asymptotic growth in the required number of qubits as a function of the problem dimensions. Section 5 introduces related work. Finally, Section 6 summarizes the findings and presents several future work directions.

2 Problem Statement

We introduce the materialized view selection problem and related terminology.

Definition 2.1 (Candidate Views). We assume that a set $V = \{v_i\}_{i \in I}$ of candidate views is available. Each view is characterized by an associated SQL query (not explicitly modeled) and by the associated storage consumption on disk, s_i , when materializing the view.

Definition 2.2 (Processing Plan). We assume that a set $Q = \{q_j\}_{j \in J}$ of queries is available. A processing plan for those queries is a function $M : Q \mapsto V \cup \{None\}$ that maps each query to at most one view to be used for processing. The storage consumption of a plan is given as the sum of storage consumptions over all views used by at least one query, i.e., $\sum_i |\exists q_j : M(q_j) = v_i| s_i$.

Note that this model allows queries to use only up to one specific view. This is consistent with prior work on materialized view selection [15], focusing on problem classes where this restriction applies.

Definition 2.3 (Cost Model). A cost model assigns each processing plan to estimated processing costs. We assume that c_j designates the cost of processing Query q_j without using any views and

Table 1: Overview of variables.

Variable	Semantics
G_i	Whether view i is generated
U_{ij}	Whether view i is used for query j
B_j	Whether query j is processed without views
S_k	Whether storage chunk k is filled
A_{ij}	Auxiliary variable for view i and query j

$c_{ij} \leq c_j$ models the cost of processing Query q_j using view v_i . The cost $Cost(M, q_j)$ of a processing plan M for a query q_j is given as $Cost(M, q_j) = c_j$ if $M(q_j) = None$ and as $Cost(M, q_j) = c_{ij}$ if $M(q_j) = v_i$. Processing costs for the entire plan are given as $\sum_j Cost(M, q_j)$.

If a View v_i is not useful to reduce processing overheads for a specific Query q_j , we set $c_{ij} = c_j$. Finally, we introduce the problem that this paper focuses on.

Definition 2.4 (Materialized View Selection). Given a set of candidate views, a query workload with an associated cost model, and a threshold T on storage capacity for materialized views, find the processing plan with minimal processing costs whose storage consumption is at or below the threshold.

The following example illustrates the problem model.

Example 2.5. Assume a set of candidate views $V = \{v_1, v_2\}$ with storage consumption $s_1 = 5$ and $s_2 = 20$ (e.g., measured in GB) that can be used for queries $Q = \{q_1, q_2\}$. View v_1 reduces execution cost from 10 to 5 (e.g., measured in seconds) for Query q_1 (i.e., $c_1 = 10$ and $c_{1,1} = 5$) but cannot be applied for the second query. View v_2 reduces processing costs for the second query from 20 to 1 (i.e., $c_2 = 20$ and $c_{2,2} = 1$), but cannot be applied for the first query. Given a storage consumption threshold of $T = 10$ for materialized views, the optimal solution generates only the first view (since the second view consumes more storage than available), maps the first query to the first view $M(q_1) = v_1$, and the second query to $None$ $M(q_2) = None$, meaning that no view is used. The aggregate processing costs for both queries are therefore $5 + 20 = 25$.

3 QUBO Transformation

This section describes how to transform instances of materialized view selection into semantically equivalent QUBO instances. The optimal solution to the QUBO problem represents an optimal choice of materialized views. Section 3.1 describes the QUBO problem variables, Section 3.2 discusses terms that appear in the QUBO formula, and Section 3.3 shows how to weight QUBO terms to ensure that optimal QUBO solutions map to optimal view choices indeed.

3.1 QUBO Variables

A QUBO problem instance contains binary variables. Next, we discuss the variables that appear in our QUBO instances. Table 1 summarizes the variables introduced next and their semantics. Table 2 describes the constants used in this and the next subsections.

Table 2: Overview of constants.

Constant	Semantics
m_i	Storage used by view i
c_j	Cost of processing query j from base data
c_{ij}	Cost of processing query j using view i
T	User-defined storage threshold
d_k	k -th chunk of disk storage
K	Highest index used for storage chunks

First, we introduce binary variables G_i for each candidate view i , determining whether the view is generated ($G_i = 1$) or not generated ($G_i = 0$). The number of such variables is directly proportional to the number of view candidates.

Queries can be processed using different views (or using base tables directly). We introduce binary variables of the form U_{ij} , representing the decision to use view candidate i to process query j . We set $U_{ij} = 1$ if view i is used to answer query j . The same view may be used to answer multiple queries. On the other hand, as discussed in Section 2, each query may exploit at most one view. Queries may be answered by processing base tables directly, without using any views. For each query j , we introduce a variable B_j , representing the possibility of processing the query without exploiting any views. If $B_j = 1$, then the query is processed without using any views.

We must be able to represent the total storage consumed by all selected view candidates. This is required to ensure that the current choice of candidate views does not exceed the user-defined storage bounds. We therefore introduce binary variables of the form S_k . Each variable S_k is associated with a weight d_k , representing a chunk of disk space. To represent all possible storage consumption values with a minimal number of variables, we generally use powers of two with $d_k = 2^k$ such that all values from zero to $2^K - 1$ can be represented using variables S_0 to S_K . If the k -th disk space chunk is used, we set the associated variable to one ($S_k = 1$), otherwise to zero. The number of variables introduced is sufficient to represent all storage consumption values up to the user-defined threshold. Higher values cannot be represented anymore, meaning that any choice of views that exceeds the storage bound leads to penalty terms, as discussed in more detail in the next subsection.

Finally, we introduce auxiliary variables A_{ij} for each candidate view i and query j . The semantics of these variables is explained in the following subsection.

Example 3.1. We discuss the variables to introduce to represent the problem described in Example 2.5. First, since we have two candidate views, we introduce variables G_1 and G_2 representing, for each of the two views, the decision of whether to generate the view or not. Second, given that we have two queries, we introduce two variables B_1 and B_2 , representing for each query the decision of whether or not to process the query from base tables (as opposed to using views). Also, we introduce variables U_{ij} for $1 \leq i \leq 2$ and $1 \leq j \leq 2$, representing the choice of using View j for answering Query i respectively. To support the constraint terms introduced in

the next section, we also need to introduce two auxiliary variables A_1 and A_2 (their semantics are explained in more detail in the following paragraphs). Finally, to represent storage consumption, we introduce variables S_0 to S_3 with associated weights 1 to $2^3 = 8$.

The optimal solution, discussed in Example 2.5, is represented by the value assignment $B_1 = 0$, $B_2 = 1$ (i.e., only the second query is processed directly from the base data) while settings $U_{11} = 1$ but $U_{12} = U_{21} = U_{22} = 0$. This captures the decision of using the first view to answer the first query, but not using any of the other views or using views for the other query.

3.2 QUBO Terms

The optimal solution to a QUBO problem minimizes the sum of terms in the QUBO formula. Hence, we must ensure that sub-optimal or invalid solutions increase the value of the QUBO formula. Next, we discuss the terms that appear in the QUBO formula.

First, we cannot use views without generating them. Hence, we must introduce terms that represent this constraint, taking high values if the constraint is violated. For each candidate view i and query j , we add terms of the form

$$(U_{ij} - G_i + A_{ij})^2 \quad (1)$$

to our QUBO formula. This term takes a minimal value of zero if either $U_{ij} = G_j = A_{ij} = 0$ (i.e., the view is not generated and not used), or $U_{ij} = 0$, $G_i = A_{ij} = 1$ (i.e., the view is generated but not used for the specific query), or $U_{ij} = G_i = 1$ while $A_{ij} = 0$ (i.e., the view is generated and used). However, if the view is used but not generated, an inconsistent solution represented as $U_{ij} = 1$ while $G_i = 0$, either setting for the auxiliary variable A_{ij} leads to a value for the term of at least one.

Next, we must ensure that each query is processed. There are two options to process a query: either by using a view or by processing base tables. The first option is expressed by setting the corresponding variable $U_{ij} = 1$ for view i and query j . The second option is captured by setting $B_j = 1$ for the associated query. The term

$$(1 - B_j - \sum_i U_{ij})^2 \quad (2)$$

takes a minimal value of zero iff either B_j or one of the variables U_{ij} is set to one. Otherwise, if none of the options for processing the query is selected, or if a conflicting set of processing options is chosen (note that each query can exploit at most one view in our problem model), the term takes values of at least one.

Finally, the QUBO formula must incorporate user-defined storage constraints. This is accomplished using a term of the following form:

$$(T' + (\sum_i m_i G_i) - (\sum_k d_k S_k))^2 \quad (3)$$

In this term, m_i is the storage consumed by view i , d_k is a chunk of storage capacity on disk. Our goal is to obtain a term for the QUBO formula that takes a minimal value of zero iff the storage consumed by all generated views does not exceed the storage threshold. The component $(\sum_i m_i G_i)$ measures the storage space consumed by all generated views on disk. The term $(\sum_k d_k S_k)$ represents the amount of storage available. Given a user-defined storage threshold

T , we choose the number of storage chunks, K , as the minimal integer such that $2^{K+1} - 1 \geq T$. Since we choose $d_k = 2^k$, it is $\sum_{0 \leq k \leq K} d_k = 2^{K+1} - 1$. Hence, by setting a suitable number of storage variables S_k to one, we can “match” any storage consumption up to $2^{K+1} - 1$.

However, we choose K to enable $2^{K+1} - 1 \geq T$. If $2^{K+1} - 1 > T$, then the term $((\sum_i m_i G_i) - (\sum_k d_k S_k))^2$ would be too “permissive”, taking a minimal value of zero even for a storage consumption above the user-defined threshold T . To disallow any storage consumption with values greater than T , we add the value $T' = 2^{K+1} - 1 - T$ in the final version of the term (Eq. 3), thereby ensuring that each storage consumption above the threshold prevents the term from taking a minimal value of zero.

All terms discussed previously represent problem constraints. Finally, we add terms that represent the objective function to minimize. Our goal is to minimize processing overheads when processing the query workload. The term

$$(\sum_j c_j B_j) + (\sum_{i,j} c_{ij} U_{ij}) \quad (4)$$

sums up processing overheads for all queries, taking into account the different options (with or without a view) for processing each query. The constants c_j represent the cost of processing query j from base tables, whereas the constants c_{ij} represent the cost of processing query j using candidate view i . Adding this term to the QUBO formula encourages solutions that minimize processing overheads. Table 3 summarizes the QUBO terms discussed in this subsection.

Example 3.2. For our running example (see Examples 2.5 and 3.1), we introduce terms of the form $(U_{ij} - G_i - A_{ij})^2$ for $1 \leq i, j \leq 2$. They represent the need to generate each of the two views before using them on specific queries. Next, we introduce terms of the form $(1 - B_j - (\sum_i U_{ij}))^2$ for $1 \leq i, j \leq 2$, representing the need to process each of the two queries using either one of the two views or no view at all. We introduced variables S_0 to S_3 , representing storage chunks ranging from size $2^0 = 1$ to size $2^3 = 8$. Summing over all storage chunk sizes, we obtain $\sum_{0 \leq k \leq 3} 2^k = 15$ as the maximal storage capacity that can be represented. This is higher than the user-defined storage threshold of $T = 10$. Hence, we add a term of the form $(5 + (\sum_i m_i G_i) + (\sum_k d_k S_k))^2$. This term can take a minimal value of zero only if the storage space consumed by all materialized views does not exceed the threshold of 10. Finally, we add the term $10B_1 + 20B_2 + 5U_{11} + 10U_{21} + 20U_{21} + 1U_{22}$, summing over different options of answering each query.

3.3 QUBO Weights

The final QUBO formula contains the terms from Table 3. Simply adding up terms is insufficient to guarantee that optimal QUBO solutions represent valid and optimal solutions to the original materialized view selection problem. For instance, when simply adding up terms, an optimal QUBO solution may choose not to process certain queries since the cost savings of not processing a specific query outweigh the penalty for not processing the query. To guarantee that optimal QUBO solutions represent valid view choices, we need to weigh the terms in the QUBO formula.

Table 3: Overview of QUBO terms.

Term	Symbol	Semantics
$(U_{ij} - G_i + A_{ij})^2$	Q_G	Must create view to use
$(1 - B_j - \sum_i U_{ij})^2$	Q_P	Must process query
$(T' + (\sum_i m_i G_i) - (\sum_k d_k S_k))^2$	Q_S	Enforce storage bound
$(\sum_j c_j B_j) + (\sum_{i,j} c_{ij} U_{ij})$	Q_C	Count processing cost

The following equation is the final QUBO formula, integrating the terms from Table 3:

$$W \cdot (Q_G + Q_P + Q_S) + Q_C \quad (5)$$

Weight W weighs all terms associated with problem constraints. Violating any of those constraints leads to an invalid solution. Each of the constraint terms (Q_G , Q_P , and Q_S) takes a value of at least one in case of a constraint violation. We must set W high enough such that a value of one in any of the constraint terms forcibly leads to a sub-optimal QUBO solution.

The total value range of the cost-related term, Q_C , ranges from zero to the sum of processing costs over all queries and processing options, i.e.:

$$\Delta = (\sum_j c_j) + (\sum_{i,j} c_{ij}) \quad (6)$$

Hence, choosing $W > \Delta$ guarantees that changing the variables to optimize the term related to processing costs, Q_C , can never make up for constraint violations.

Example 3.3. For our running example, we can use the following weights to ensure that solutions satisfying all problem constraints are prioritized. Specifically, we sum up cost values of all processing options for each query, namely $10 + 5 + 20 + 1 = 36$, and add one to obtain a suitable weight (37).

4 Formal Analysis

Before analyzing the properties of the QUBO transformation, we first analyze the hardness of the specific variant of materialized selection that it refers to.

THEOREM 4.1. *Materialized view selection, as defined in this paper, is NP-hard.*

PROOF. The proof uses a reduction from the knapsack problem. Given an instance of the knapsack problem, defined by items $\mathcal{I}$ with weights $\mathcal{W}_i$ and utility $\mathcal{U}_i$, and a weight threshold $\mathcal{T}$, we introduce an equivalent instance of materialized view selection (according to Definition 2.4). Introduce one view v_i for each item i and an associated query q_i . Set $c_{ij} = c_j$ for any $i \neq j$. For $i = j$, the cost savings due to using the view should be proportional to the benefit of the corresponding item. This means that each view is only useful for one single query. Also, set the storage space consumption of each view to the weight of the corresponding item ($s_i = \mathcal{W}_i$) and the storage threshold to the weight threshold ($T = \mathcal{T}$). The optimal solution to the materialized view selection problem translates into

Table 4: Overview of problem dimensions.

Symbol	Semantics
n_V	Number of view candidates
n_Q	Number of queries to process
T	Storage threshold imposed by users

an optimal solution to the knapsack problem (by selecting the items associated with generated views). $\square$

This is consistent work with prior work [2, 22], showing that multiple other variants of materialized view selection are NP-hard. This hardness result justifies the use of sophisticated optimization hardware. Next, we analyze the properties of the QUBO transformation introduced in the previous section.

The primary bottleneck in current quantum hardware is the limited number of qubits. Hence, approaches for mapping problems to QUBO formulas are often analyzed in terms of their asymptotic qubit complexity. This metric captures the asymptotic rate at which the number of required qubits grows as a function of the problem dimensions. For the following analysis, we measure the problem size as a function of the problem dimensions reported in Table 4.

First, we analyze the asymptotic growth of the number of (logical) problem variables as a function of the problem size. Table 1 gives an overview of the variables that appear in the QUBO formula.

THEOREM 4.2. *The number of problem variables in the QUBO formula is in $O(n_V \cdot n_Q + \log(T))$.*

PROOF. We introduce one variable G_i per view, $O(n_V)$ variables in total. Also, we introduce one variable B_j per query, $O(n_Q)$ variables in total. The number of variables representing view generation (G_i) and query processing without views (B_j) is dominated by the number of variables U_{ij} representing the option to use a specific view for a specific query, and the number of auxiliary variables A_{ij} (used in terms linking view usage to view generation). For both variable sets, the number of variables is in $O(n_V \cdot n_Q)$. Finally, the number of variables S_k representing storage consumption is logarithmic in the size of the storage threshold T (since we introduce weights that follow a geometric progression): $O(\log(T))$. $\square$

The number of problem variables constitutes a lower bound on the number of required qubits. For certain accelerators that solve QUBO problems, only one qubit is required to represent each problem variable, indeed. For instance, for digital annealers (such as Fujitsu’s digital annealer), all qubits are mutually connected. This means that quadratic terms involving any combination of problem variables can be represented while using just a single qubit per variable.

On the other hand, for current quantum annealer models (e.g., the quantum annealers by D-Wave), the connectivity between qubits is sparse. Each qubit is connected only to a constant number of neighboring qubits. As only connected qubits can represent quadratic terms, the number of quadratic terms therefore implies another lower bound on the required number of qubits. Next, we calculate

the number of quadratic terms in our QUBO formula. We refer to the symbols for each QUBO term introduced in Table 3.

THEOREM 4.3. *The QUBO formula introduces $O(n_Q \cdot n_V^2 + \log(T)^2 + \log(T) \cdot n_V)$ products between variables.*

PROOF. Term Q_G , added for each combination of a view and a query, introduces quadratic terms connecting variables U_{ij} , G_i , and A_{ij} for fixed values of i and j . This introduces $O(n_V \cdot n_Q)$ quadratic terms in total. Term Q_P , added for each query, adds products between all variables U_{ij} and B_j for a fixed value of j . Hence, it adds $O(n_V^2)$ products for each query, and therefore $O(n_Q \cdot n_V^2)$ products in total. Term Q_S introduces products of the form $S_{k_1} \cdot S_{k_2}$ between variables representing storage consumption, products of the form $G_{i_1} \cdot G_{i_2}$ connecting variables representing the generation of different views, and products of the form $G_i \cdot S_k$ (linking variables representing view generation to variables representing storage consumption). In total, this introduces $O(\log(T)^2 + n_V^2 + n_V \cdot \log(T))$ products between variables (taking into account the number of variables in different categories, as discussed in the proof of Theorem 4.2). Term Q_C does not contain any products between variables. Summing up the number of products over variable pairs in different categories (while discarding dominated terms) yields the final result. $\square$

This immediately implies the following corollary.

COROLLARY 4.4. *On sparse annealer architecture, connecting each qubit only to a constant number of other qubits, the number of required qubits is in $\Omega(n_Q \cdot n_V^2 + \log(T)^2 + \log(T) \cdot n_V)$.*

This follows immediately from Theorem 4.3 since each quadratic term needs to be realized via a coupling and the number of available coupling per qubit is a constant on sparse annealer architectures.

5 Related Work

This paper connects to a large body of prior work in the database community targeting the problem of materialized view selection [1, 2, 15], or related problems such as index selection [7, 8, 32]. However, most prior work uses classical hardware for optimization. They are therefore not applicable to exploit the capabilities of quantum accelerators.

By its focus on quantum hardware, this paper relates to prior work in the database community, aimed at exploiting quantum accelerators to solve hard problems in data management [10, 12, 30] or to apply ideas and systems originating in the database community to solve problems related to quantum computing [6, 14, 29].

In particular, this paper relates to one prior work [19] exploiting quantum evolutionary algorithms to solve materialized view selection. However, the latter paper focuses on gate-based quantum computing and is not applicable to map materialized view selection to quantum annealers. At the time of writing, the latter category of hardware accelerators feature significantly more qubits than gate-based quantum hardware, making quantum annealing more applicable to problems of practically relevant sizes.

More broadly, this research connects to prior work solving other hard optimization problems in data management, including multiple query optimization [10, 26, 30], join ordering [18, 24, 25, 27],

schema matching [12], index selection [17, 31], or transaction scheduling [5, 21]. Among the aforementioned problems, this work relates most to the problem of index selection, due to the problem domain (physical design optimization) and the problem structure (e.g., presence of storage bounds). However, prior work exploiting quantum accelerators for index selection either focus on gate-based hardware [17] (as opposed to quantum annealers), or use simple problem models that differ significantly from the one explored in this paper and are unsuitable for materialized view selection (e.g., associating index candidates with a utility value [31], as opposed to modeling the option of using specific views for specific queries).

6 Conclusion and Outlook

This paper describes a first approach to solving the materialized view selection problem on a quantum annealer. The proposed approach requires a number of qubits that grows quadratically in the number of candidate views. A first direction for future work is reducing the asymptotic number of required qubits (or proving that the current number is optimal).

Second, the proposed approach is only evaluated from the formal perspective in this paper. Experiments with actual quantum hardware, as well as quantum-inspired accelerators such as digital annealers, are needed to assess how well the proposed approach works in practice. A comparison with algorithms executed on classical computing hardware will reveal whether or not the proposed approach is competitive for certain problem classes.

References

- [1] 1997. Selection of views to materialize in a data warehouse. In *ICDT*. 98–112. doi:10.1007/3-540-62222-5_39
- [2] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated selection of materialized views and indexes in SQL databases. In *VLDB*. 496–505. <http://portal.acm.org/citation.cfm?id=671701>
- [3] Dorit Aharonov, Wim Van Dam, Julia Kempe, Zeph Landau, Seth Lloyd, and Oded Regev. 2007. Adiabatic quantum computation is equivalent to standard quantum computation. In *SIAM Journal on Computing*, Vol. 37. 166–194. Issue 1. doi:10.1137/S0097539705447323
- [4] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. Divincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. 1995. Elementary gates for quantum computation. *Physical Review A* 52 (1995), 3457–3467. Issue 5. doi:10.1103/PhysRevA.52.3457
- [5] Tim Bittner and Sven Groppe. 2020. Avoiding blocking by scheduling transactions using quantum annealing. In *Proceedings of the 24th Symposium on International Database Engineering & Applications* (New York, NY, USA). ACM, 1–10. doi:10.1145/3410566.3410593
- [6] Mark Blacher, Julien Klaus, Christoph Staudt, Sören Laue, Viktor Leis, and Joachim Giesen. 2023. Efficient and Portable Einstein Summation in SQL. *Proceedings of the ACM on Management of Data* 1 (6 2023), 1–19. Issue 2. doi:10.1145/3589266
- [7] Alberto Caprara, Dario Maio, and Matteo Fischetti. 1995. Exact and Approximate Algorithms for the Index Selection Problem in Physical Database Design. *IEEE Transactions on Knowledge and Data Engineering* 7 (1995), 955–967. Issue 6. doi:10.1109/69.476501
- [8] Surajit Chaudhuri. 2004. Index selection for databases: A hardness study and a principled heuristic solution. *KDE* 16 (2004), 1313–1323. Issue 11. doi:10.1109/TKDE.2004.75 Problem* Idea: "Model index selection as knapsack problem" Assign index to benefit using LP formulation* Allows to reduce error of neglecting interactionSome restrictions - focus on interactions between pairs of indices?.
- [9] G. Chiribella, G. M. D'Ariano, and P. Perinotti. 2008. Quantum circuit architecture. *Physical Review Letters* 101 (2008), 1–4. Issue 6. doi:10.1103/PhysRevLett.101.060401
- [10] Tobias Fankhauser, Marc E. Solèr, Rudolf Marcel Fuchsli, and Kurt Stockinger. 2023. Multiple Query Optimization Using a Gate-Based Quantum Computer. *IEEE Access* 11 (2023), 114031–114043. doi:10.1109/ACCESS.2023.3324253
- [11] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. 2000. Quantum computation by adiabatic evolution. *CoRR* abs/000110 (2000), 1–24. doi:quant-ph/0001106
- [12] Kristin Fritsch and Stefanie Scherzinger. 2023. Solving Hard Variants of Database Schema Matching on Quantum Computers. In *PVLDB*, Vol. 16. 3990–3993. Issue 12. doi:10.14778/3611540.3611603
- [13] Hayato Goto, Kosuke Tatumura, and Alexander R. Dixon. 2019. Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems. *Science Advances* 5 (2019), 1–8. Issue 4. doi:10.1126/sciadv.aav2372
- [14] Rihan Hai, Shih-Han Hung, Tim Coopmans, Tim Littau, and Floris Geerts. 2025. Quantum Data Management in the NISQ Era. *Proceedings of the VLDB Endowment* 18 (2 2025), 1720–1729. Issue 6. doi:10.14778/3725688.3725701
- [15] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing data cubes efficiently. In *SIGMOD*. 205–216. doi:10.1145/235968.233333
- [16] Yu-Ting Kao, Jia-Le Liao, and Hsiu-Chuan Hsu. 2023. Solving Combinatorial Optimization Problems on Fujitsu Digital Annealer. *CoRR* abs/2311.0 (2023), 1–5. <http://arxiv.org/abs/2311.05196>
- [17] Manish Kesarwani and Jayant R. Haritsa. 2024. Index Advisors on Quantum Platforms. *Proceedings of the VLDB Endowment* 17 (2024), 3615–3628. Issue 11. doi:10.14778/3681954.3682025
- [18] Hanwen Liu, Federico Spedalieri, and Ibrahim Sabek. 2025. A Demonstration of Q 2 O: Quantum-Augmented Query Optimizer. *Proceedings of the VLDB Endowment* 18 (8 2025), 5439–5443. Issue 12. doi:10.14778/3750601.3750691
- [19] Raouf Mayata and Abdelmadjid Boukra. 2020. Using quantum evolutionary based algorithm to solve materialized view selection problem. In *Proceedings of the 2nd International Conference on Digital Tools & Uses Congress*. 1–8.
- [20] Catherine C. McGeoch and Cong Wang. 2013. Experimental evaluation of an adiabatic quantum system for combinatorial optimization. In *Proceedings of the ACM International Conference on Computing Frontiers, CF 2013*. doi:10.1145/2482767.2482797
- [21] Nitin Nayak, Alexandru Prisacaru, Umut Çalikylmaz, Jinghua Groppe, and Sven Groppe. 2025. Quantum-Enhanced Transaction Scheduling with Reduced Complexity via Solving QUBO Iteratively using a Locking Mechanism. In *Proceedings of the 2nd Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications* (New York, NY, USA). ACM, 26–35. doi:10.1145/3736393.3736701
- [22] Yue Pang, Lei Zou, Jeffrey Xu Yu, and Linglin Yang. 2024. Materialized View Selection and View-Based Query Planning for Regular Path Queries. *Proceedings of the ACM on Management of Data* 2 (5 2024), 1–26. Issue 3. doi:10.1145/3654955
- [23] D. Pathania, S. Momose, T. Nishimura, and M. Ikuta. 2022. Accelerating and Optimizing Oil and Gas Exploration Planning Using Quantum Inspired Classical Computing or Vector Annealing. In *Sixth EAGE High Performance Computing Workshop*. 1–7. doi:10.3997/2214-4609.2022615018
- [24] Pranshi Saxena, Ibrahim Sabek, and Federico Spedalieri. 2024. Constrained Quadratic Model for Optimizing Join Orders. In *Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications* (New York, NY, USA). ACM, 38–44. doi:10.1145/3665225.3665447
- [25] Manuel Schoenberger, Immanuel Trummer, and Wolfgang Mauerer. 2023. Quantum-Inspired Digital Annealing for Join Ordering. *PVLDB* 17 (2023), 511–524. Issue 3. doi:10.14778/3632093.3632112
- [26] Manuel Schoenberger, Immanuel Trummer, and Wolfgang Mauerer. 2026. Large-Scale Multiple Query Optimisation with Incremental Quantum-(Inspired) Annealing. *SIGMOD* 3 (2026), 1–25. Issue 4. doi:10.1145/3749171
- [27] Manuel Schönberger, Stefanie Scherzinger, and Wolfgang Mauerer. 2023. Ready to Leap (by Co-Design)? Join Order Optimisation on Quantum Hardware. *Proceedings of the ACM on Management of Data* 1 (2023), 1–27. Issue 1. doi:10.1145/3588946
- [28] Shriyank Somvanshi, Subash Das, Md Monzurul Islam, Sazzad Bin Bashir Pollock, Gaurab Chhetri, and Darrell Anderson. 2026. Quantum Computing in Transportation Engineering: A Survey. *IEEE Transactions on Intelligent Transportation Systems* (2026), 1–18. doi:10.1109/TITS.2026.3662626
- [29] Immanuel Trummer. 2024. Towards Out-of-Core Simulators for Quantum Computing. In *Q-Data*. 1–1. doi:10.1145/3665225.3665441
- [30] Immanuel Trummer and Christoph Koch. 2016. Multiple Query Optimization on the D-Wave 2X Adiabatic Quantum Computer. *PVLDB* 9 (2016), 648–659. Issue 9. doi:10.14778/2947618.2947621
- [31] Immanuel Trummer and Davide Venturelli. 2024. Leveraging Quantum Computing for Database Index Selection. In *Q-Data*. 14–26. doi:10.1145/3665225.3665445
- [32] Gary Valentini, M. Zuliani, Daniel C. Zilio, Guy M Lohman, and Alan Skelley. 2000. DB2 advisor: An optimizer smart enough to recommend its own indexes. In *ICDE*. 101–110. doi:10.1109/ICDE.2000.839397
- [33] Supreeth Mysore Venkatesh, Antonio Macaluso, Marlon Nuske, Matthias Klusch, and Andreas Dengel. 2024. Quantum Annealing-Based Algorithm for Efficient Coalition Formation Among LEO Satellites. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 205–210. doi:10.1109/QCE60285.2024.10279
- [34] Liang Zhang, Yin Xu, Mohan Wu, Liang Wang, and Hua Xu. 2026. Quantum long short-term memory for drug discovery. *EPJ Quantum Technology* 13 (12 2026), 14. Issue 1. doi:10.1140/epjqt/s40507-026-00467-1